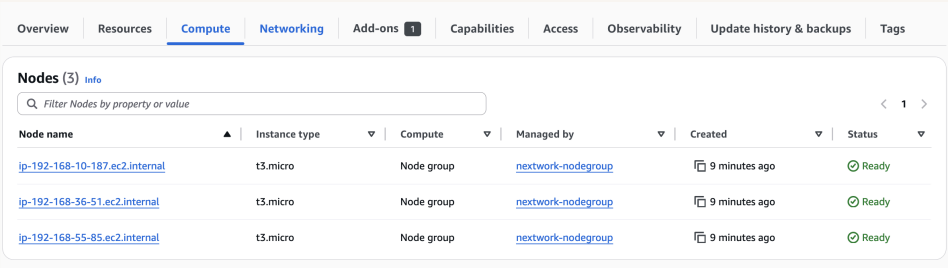
nextwork.org

Launch a Kubernetes Cluster

CH Arpita Chowdhury



CH
Arpita Chowdhury
NextWork Student

Introducing Today's Project!

In this project, I will architect, automate, and deploy a scalable cloud environment using Terraform and Kubernetes on AWS, focusing on reliability, security, and real-world constraints. I am doing this because I am preparing for professional cloud engineering roles where infrastructure is treated as code, systems must scale predictably, and downtime is not acceptable.

CH
Arpita Chowdhury
NextWork Student

What is Kubernetes?

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. Companies and developers use Kubernetes to reliably run applications at scale, handle traffic changes automatically, improve availability through self-healing systems, and manage complex microservices environments across cloud and on-premises infrastructure.

I used eksctl to create and manage an Amazon EKS cluster from the command line, simplifying the setup of Kubernetes infrastructure on AWS. The create cluster command I ran defined key cluster settings such as the cluster name, AWS region, Kubernetes control plane, worker node groups, networking configuration, and the IAM roles required for the cluster to operate securely.

I initially ran into two errors while using eksctl. The first one was because I installed eksctl in the wrong place (on my local machine instead of inside the EC2 instance), so the EC2 environment didn’t have the tool available to run EKS commands. The second one was because my EC2 instance did not yet have the correct IAM role/permissions to call the AWS APIs that eksctl uses (especially CloudFormation and EKS actions), which resulted in an AccessDenied/authorization-style error when trying to create the cluster.

Arpita Chowdhury
NextWork Student

CH
Arpita Chowdhury
NextWork Student

eksctl and CloudFormation

CloudFormation helped create my EKS cluster because eksctl uses it to automatically provision and manage all the required AWS infrastructure as code, ensuring the resources are created in the correct order and tracked as a single stack. It created VPC resources because an EKS cluster needs networking components such as subnets, route tables, security groups, and internet gateways to allow the control plane and worker nodes to communicate securely within AWS.

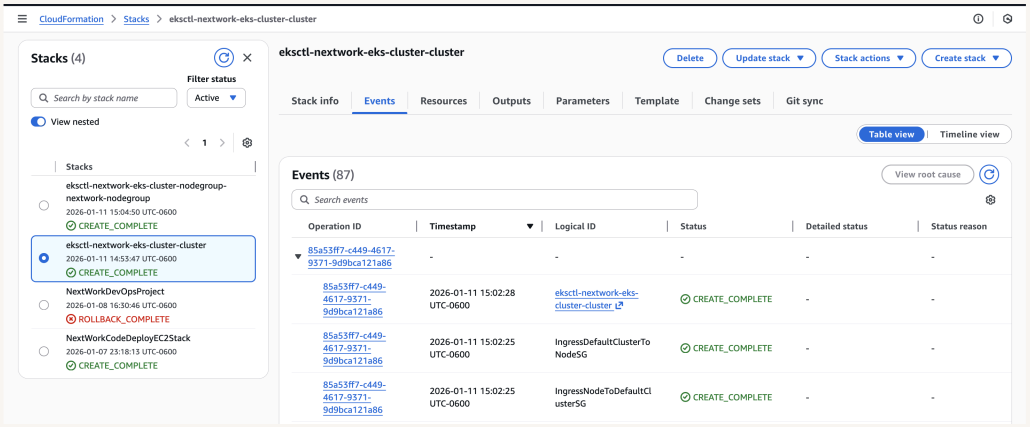
There was also a second CloudFormation stack for the EKS node group, which provisions the EC2 worker nodes, Auto Scaling group, and related IAM roles that actually run the Kubernetes workloads. The difference between a cluster and a node group is that the cluster represents the Kubernetes control plane and management layer, while the node group consists of the EC2 instances that register with the cluster and execute containerized applications.

CH

Arpita Chowdhury

NextWork Student

nextwork.org



CH
Arpita Chowdhury
NextWork Student
nextwork.org

The EKS console

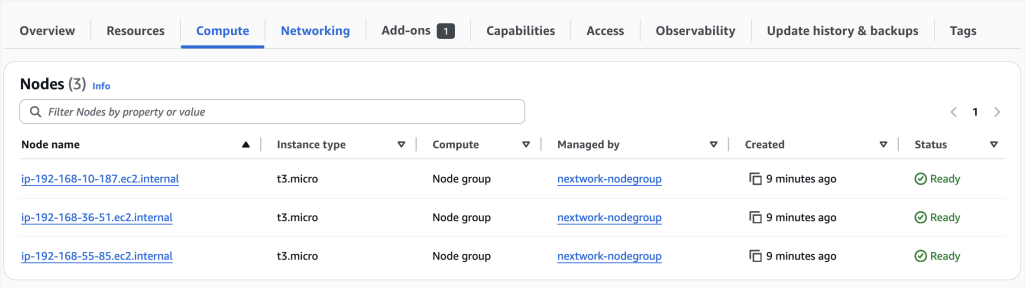
I had to create an IAM access entry in order to give my IAM user permission to access and manage the Kubernetes API of my Amazon EKS cluster. An access entry is a mapping between an AWS IAM identity (such as a user or role) and Kubernetes role-based access control (RBAC). It tells EKS which IAM principals are allowed to authenticate to the Kubernetes control plane and what level of permissions they have inside the cluster. I set it up by adding my IAM user to the EKS cluster's Access section, attaching the AmazonEKSClusterAdminPolicy, and giving it cluster-wide scope. This granted my user full administrative permissions across the entire Kubernetes cluster, allowing me to manage nodes, namespaces, workloads, and all cluster resources securely through AWS IAM and Kubernetes RBAC.

It took about 15–20 minutes to create my EKS cluster, including provisioning the control plane, setting up networking, installing core add-ons, and launching the managed node group. Most of this time was spent waiting for CloudFormation to create the VPC, subnets, security groups, and EC2 worker nodes, which are required for a production-ready Kubernetes environment. Since I will create this cluster again in later projects, this process could be sped up if I use a predefined eksctl configuration file or Infrastructure-as-Code template that already contains the cluster, node group, and networking settings. This would allow me to reuse the same setup with a single command instead of manually specifying parameters each time, making the deployment faster, more consistent, and easier to automate in future DevOps workflows.

CH

Arpita Chowdhury
NextWork Student

nextwork.org



EXTRA: Deleting nodes

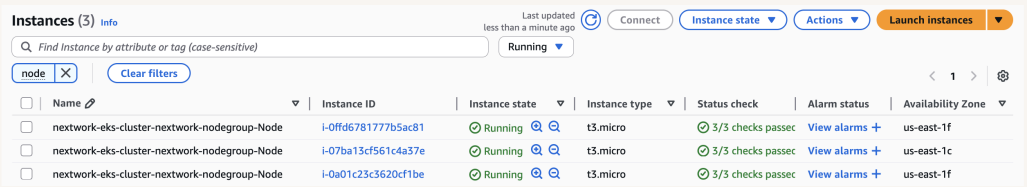
This is because EKS worker nodes are actually regular Amazon EC2 instances running a special Kubernetes-optimized operating system and software. When a node group is created, AWS launches EC2 instances on your behalf and installs the Kubernetes components (kubelet, container runtime, and networking agents) that allow them to join the EKS cluster. These EC2 instances register themselves with the EKS control plane and become Kubernetes nodes, but they are still managed and billed as EC2 instances. That is why you can see, start, stop, or terminate them from the EC2 console, while Kubernetes simultaneously sees them as nodes that run containers inside your cluster.

Desired size means the number of EC2 instances that the node group should currently be running. In this case, a desired size of 3 nodes tells AWS to keep exactly three worker nodes active in the Kubernetes cluster so that workloads always have compute capacity available. Minimum size defines the lowest number of nodes that the cluster is allowed to scale down to. This ensures the cluster never shrinks below a safe level, which is important for keeping essential system pods and applications running. Maximum size defines the highest number of nodes the cluster is allowed to scale up to. This prevents runaway scaling that could cause unexpected costs or resource overuse while still allowing Kubernetes to add more compute when application demand increases. Together, these three settings allow the cluster to automatically scale up and down based on workload demand while maintaining reliability, availability, and cost control.

placeholder

When I deleted my EC2 instances, the node group noticed that the number of running instances dropped below the desired size of three. As a result, AWS automatically created new EC2 instances and added them back into the cluster.

Kubernetes then registered the new instances as nodes and continued running workloads without manual intervention. This is because EKS integrates Kubernetes with EC2 Auto Scaling Groups. The node group enforces the desired capacity, while Kubernetes handles workload scheduling and recovery, creating a self-healing system that automatically replaces failed or terminated self-healing system that automatically replaces failed or terminated servers to keep applications running reliably.



The place to learn & showcase your skills

Check out nextwork.org for more projects